

CONTENTS

Pipelining	
Módulo 9 — Arquitectura de Computadoras (FING UdelaR) — Guía de estudio con rigor de tolerancia cero	
0. Por qué existe este problema	
1. Idea básica del pipeline	
2. Fórmula de aceleración (Speedup)	
2.1 Fórmula	
2.2 De dónde sale	
2.3 Ejemplos numéricos completos (verificados)	
2.4 Comportamiento cuando $n \rightarrow \infty$	
2.5 Tabla resumen	
3. Hazards estructurales	
Soluciones	
4. Hazards de datos	
4.1 RAW — Read After Write (lectura después de escritura)	
4.2 WAR — Write After Read (escritura después de lectura)	
4.3 WAW — Write After Write (escritura después de escritura)	
4.4 Soluciones a los hazards de datos	
4.5 Tabla resumen de hazards de datos	
5. Hazards de control	
Soluciones	

6. Ejecución fuera de orden

Ejercicios de repaso rápido (con respuesta)

Resumen ultra-rápido (2 minutos)

Pipelining

MÓDULO 9 — ARQUITECTURA DE COMPUTADORAS (FING UDELAR) — GUÍA DE ESTUDIO CON RIGOR DE TOLERANCIA CERO

0. Por qué existe este problema

Una CPU ejecuta instrucciones siguiendo el “ciclo de instrucción” clásico: **Fetch** (buscar la instrucción en memoria), **Decode** (decodificarla), **Read** (leer operandos), **Execute** (ejecutar la operación), **Write** (guardar el resultado). Si estas 5 etapas se hacen una detrás de la otra para cada instrucción, y cada etapa tarda un ciclo de reloj, entonces **una instrucción completa tarda 5 ciclos**, y hasta que esa instrucción no termina, la unidad de control ni siquiera empieza a buscar la próxima.

El problema es evidente: durante los 5 ciclos que dura la instrucción 1, el hardware de “Fetch” está ocioso 4 de esos 5 ciclos (solo lo usa en el primero), el hardware de “Decode” está ocioso el resto, etc. Es como tener una fábrica con 5 estaciones de trabajo especializadas, pero donde un solo producto recorre toda la fábrica antes de dejar entrar al siguiente: 4 de las 5 estaciones están siempre paradas.

La idea de **pipelining** (segmentación) es la misma que revolucionó la industria a principios del siglo XX (Henry Ford, línea de montaje): si divido el trabajo en etapas independientes, puedo tener **varias instrucciones distintas, cada una en una etapa distinta, avanzando simultáneamente**. Mientras la instrucción 1 está en Execute, la instrucción 2 puede estar en Read, la instrucción 3 en Decode y la instrucción 4 en Fetch — todo en el mismo ciclo de reloj.

Esto ****no**** hace que una instrucción individual se ejecute más rápido (sigue tardando 5 ciclos de principio a fin). Lo que mejora es el **throughput**: la cantidad de instrucciones que terminan por unidad de tiempo. Es la diferencia entre “tiempo de un trámite” y “cantidad de trámites atendidos por hora” en una oficina con varias ventanillas en cadena.

El resto de este módulo trata sobre: (1) cuánto mejora exactamente el rendimiento (speedup), y (2) qué situaciones concretas impiden que el pipeline funcione “de manera ideal” — los llamados **hazards** (obstáculos/peligros) — y cómo se resuelven.

1. Idea básica del pipeline

Definición. Un pipeline de instrucciones divide la ejecución de cada instrucción en k etapas (por ejemplo, Fetch–Decode–Execute–Write, $k=4$, o Fetch–Decode–Read–Execute–Write, $k=5$), implementadas cada una por un bloque de hardware distinto, sincronizadas por el mismo reloj. En cada ciclo de reloj, cada instrucción “avanza” una etapa, y una nueva instrucción entra a la etapa de Fetch.

Analogía de la línea de montaje. Si lavar-secar-doblar ropa toma 30-40-20 minutos y tengo 4 cargas de ropa, sin pipeline tardo $4 \times 90 = 360$ minutos. Con pipeline, la lavadora empieza con la carga 2 apenas la carga 1 pasa a la secadora, y así sucesivamente: se **solapan** las etapas de cargas distintas.

Diagrama de tiempo (4 instrucciones, pipeline de 4 etapas F-D-E-W):

	ciclo:	1	2	3	4	5	6	7
Instrucción 1:		F	D	E	W			
Instrucción 2:			F	D	E	W		
Instrucción 3:				F	D	E	W	
Instrucción 4:					F	D	E	W

Ejemplo 1. Con este diagrama, la instrucción 1 termina en el ciclo 4 (tardó 4 ciclos, como siempre). Pero la instrucción 4 *también* ya terminó en el ciclo 7, no en el ciclo 16 que hubiera tardado sin pipeline ($4 \text{ instrucciones} \times 4 \text{ ciclos cada una}$). En el ciclo 7 ya completé las 4 instrucciones, en vez de los 16 ciclos que tomaría de forma serial.

Truco de verificación: en un pipeline de k etapas funcionando “en régimen” (steady state, todas las etapas ocupadas), sale **una instrucción terminada por ciclo**. Contá cuántas columnas del diagrama tienen una “W” y compáralo con el número de ciclo: deberían coincidir en régimen.

Error frecuente: pensar que el pipeline hace que *cada instrucción individual* se ejecute más rápido. Es falso: cada instrucción sigue tardando k ciclos de principio a fin (su *latencia* no cambia). Lo que se reduce es el tiempo total para ejecutar *muchas* instrucciones (mejora el *throughput*, no la latencia individual).

Para que el pipeline funcione de manera óptima se necesitan dos condiciones (que en la práctica casi nunca se cumplen del todo, y de ahí surgen los hazards de las secciones 3 a 5):

1. Las etapas deben ser **independientes entre sí** (usar hardware distinto).
2. Las etapas deben tener **duración similar** (si una tarda mucho más que las demás, esa etapa se vuelve el cuello de botella y determina el ritmo de todo el pipeline).

2. Fórmula de aceleración (Speedup)

2.1 FÓRMULA

$$S = \frac{n}{n+k-1}$$

donde:

- n = cantidad de instrucciones a ejecutar
- k = cantidad de etapas del pipeline

2.2 DE DÓNDE SALE

Sin pipeline: cada instrucción se ejecuta de punta a punta antes de empezar la próxima. Cada instrucción tarda k ciclos (una por etapa), y tengo n instrucciones, así que:

$$T_{\text{sin pipeline}} = n \cdot k \text{ ciclos}$$

Con pipeline: miren el diagrama de tiempo de la sección 1. La *primera* instrucción tarda k ciclos en completarse (tiene que atravesar las k etapas, no hay atajo). Pero a partir de ahí, gracias al solapamiento, **sale una instrucción completa por cada ciclo adicional**. Como ya usé k ciclos para completar la primera instrucción, me quedan $n-1$ instrucciones más, y cada una añade solo 1 ciclo (porque todas las etapas anteriores para esa instrucción ya se solaparon con las instrucciones previas). Entonces:

$$T_{\text{con pipeline}} = k + (n-1) = n + k - 1 \text{ ciclos}$$

Truco de verificación: contá las columnas del diagrama de tiempo hasta que aparece la última “W”. Con $n=4$ y $k=4$ (ejemplo de la sección 1): $T = 4+4-1 = 7$ ciclos. Efectivamente la instrucción 4 termina en el ciclo 7. ✓

El **speedup** (aceleración) es la relación entre ambos tiempos — cuántas veces más rápido termino el lote de n instrucciones gracias al pipeline:

$$S = \frac{T_{\text{sin pipeline}}}{T_{\text{con pipeline}}} = \frac{nk}{n+k-1}$$

2.3 EJEMPLOS NUMÉRICOS COMPLETOS (VERIFICADOS)

Ejemplo 1: $n=100$ instrucciones, $k=5$ etapas.

Sin pipeline: $T = n \cdot k = 100 \cdot 5 = 500$ ciclos.

Con pipeline: $T = n+k-1 = 100+5-1 = 104$ ciclos.

Speedup: $S = \frac{100 \cdot 5}{100+5-1} = \frac{500}{104} = 4.8077$

Verificación numérica (calculado con Python): $500/104 = 4.8076923\dots$ ✓. Noten que $S=4.81$ está cerca de $k=5$ pero no llega, porque todavía hay una penalización relativa

por el “llenado” inicial del pipeline (los primeros $k-1$ ciclos donde no sale ninguna instrucción completa todavía).

Ejemplo 2: $n=1000$ instrucciones, $k=5$ etapas (mismo pipeline, más instrucciones).

Sin pipeline: $T = 1000 \times 5 = 5000$ ciclos.

Con pipeline: $T = 1000 + 5 - 1 = 1004$ ciclos.

Speedup: $S = \frac{5000}{1004} = 4.9801$

Verificación numérica: $5000/1004 = 4.9800796\dots$ ✓. Comparado con el Ejemplo 1, al aumentar n el speedup se acercó más a 5 (de 4,81 a 4,98). Esto ilustra el comportamiento asintótico que se explica a continuación.

2.4 COMPORTAMIENTO CUANDO $n \rightarrow \infty$

Si n es mucho mayor que k ($n \gg k$), el término $k-1$ del denominador se vuelve despreciable frente a n :

$$S = \frac{nk}{n+k-1} \xrightarrow{n \rightarrow \infty} \frac{nk}{n} = k$$

El speedup máximo teórico que puede dar un pipeline es igual a su número de etapas k .

Nunca se puede superar ese límite (con un único pipeline sin otras técnicas como superescalaridad), y solo se alcanza aproximadamente cuando se ejecutan muchísimas instrucciones sin interrupciones del flujo.

Verificación con una tabla de convergencia (pipeline de $k=5$ etapas, calculado con Python):

n	$S = nk/(n+k-1)$
1	1,0000
8	2,9091
100	4,8077
1000	4,9801
10000	4,9980

A medida que n crece, S se acerca cada vez más a $k=5$ sin nunca superarlo. ✓

Error frecuente en el parcial: escribir la fórmula como $S = \frac{nk}{n+k}$ (sin el -1) o como $S = \frac{n+k-1}{nk}$ (invertida). Para no equivocarse: recordá que el speedup siempre tiene que dar un número **mayor o igual a 1** (el pipeline nunca puede ser más lento que la ejecución serial) y **menor o igual a k** (nunca puede superar la cantidad de etapas). Si al reemplazar valores te da $S > k$ o $S < 1$, tenés la fórmula mal escrita.

Otra forma equivalente y muy usada de escribir lo mismo es en términos de *tiempo* T por etapa: $T_{\text{sin pipeline}} = nkT$ y $T_{\text{con pipeline}} = (n+k-1)T$, donde el factor T (duración de una etapa/ciclo de reloj) se cancela al hacer el cociente. Por eso el speedup no depende de T , solo de n y k .

2.5 TABLA RESUMEN

Magnitud	Fórmula
Tiempo sin pipeline	$T_{\text{sin}} = nk$ ciclos
Tiempo con pipeline	$T_{\text{con}} = n+k-1$ ciclos
Speedup	$S = \frac{nk}{n+k-1}$
Speedup máximo ($n \rightarrow \infty$)	$S \rightarrow k$
Rango válido de S	$1 \leq S \leq k$

3. Hazards estructurales

Definición. Un hazard estructural ocurre cuando **dos etapas distintas del pipeline, en el mismo ciclo, necesitan el mismo recurso de hardware** y ese recurso no puede atender a ambas simultáneamente.

Ejemplo típico. Si la memoria es única (no hay arquitectura Harvard con memorias separadas para datos e instrucciones), puede pasar que en un mismo ciclo la etapa de *Fetch* de la instrucción 3 necesite leer memoria para traer la próxima instrucción, mientras que la etapa de *Read* (o *Memory*) de la instrucción 1 también necesita acceder a memoria para leer un operando. Ambas quieren usar el mismo bus de memoria al mismo tiempo: conflicto.

Otro ejemplo. Si la etapa de *Fetch* necesita sumar 4 al PC (Program Counter) para calcular la dirección de la próxima instrucción, y para eso usa la ALU, puede chocar con la etapa de

Execute de otra instrucción que en ese mismo ciclo también necesita la ALU para hacer una operación aritmética.

SOLUCIONES

1. **Burbuja / stall (esperar):** se inserta artificialmente una “instrucción nula” (NOP, No Operation) que no requiere ningún recurso, deteniendo el avance de la instrucción que perdió la pulseada por el recurso durante un ciclo. Es la solución más simple, pero cuesta rendimiento (throughput perdido).
2. **Duplicar el hardware:** se agrega un segundo recurso independiente para que ambas etapas puedan trabajar en paralelo sin pelearse. Ejemplos reales: usar una **arquitectura Harvard** (memoria de instrucciones separada de la memoria de datos, así Fetch y Read/Memory no compiten) o agregar un **sumador dedicado** para calcular PC+4 en la etapa de Fetch, en vez de compartir la ALU con la etapa de Execute.

Error frecuente: confundir un hazard estructural con un hazard de datos. La clave para distinguirlos: el hazard estructural es un problema de **hardware** (dos etapas peleando por el mismo circuito físico, sin importar qué datos estén usando), mientras que el hazard de datos es un problema de **valores** (una instrucción necesita un dato que otra todavía no calculó), aunque el hardware alcance de sobra.

4. Hazards de datos

Definición. Ocurren cuando hay **dependencias de datos entre instrucciones** que están simultáneamente dentro del pipeline: el orden en que el hardware terminaría de leer/escribir esos datos (por el solapamiento de etapas) no coincide con el orden lógico que el programa necesita.

Hay tres tipos, según qué combinación de lectura/escritura se ve afectada:

4.1 RAW — READ AFTER WRITE (LECTURA DESPUÉS DE ESCRITURA)

El más común y el más problemático. Ocurre cuando una instrucción posterior necesita **leer** un operando que una instrucción anterior todavía **no terminó de escribir** (porque esa escritura ocurre en una etapa posterior del pipeline, cuando la instrucción anterior aún no llegó a la etapa Write).

Ejemplo concreto (verificado):

```
I1:  ADD  R1, R2, R3      ; R1 := R2 + R3
I2:  SUB  R4, R1, R5      ; R4 := R1 - R5
```

I2 necesita leer R1 como operando origen. Pero R1 todavía no fue escrito por I1: I1 recién calcula R1 en su etapa *Execute* y lo guarda en su etapa *Write*, varios ciclos después de que I2 ya pasó por su propia etapa de lectura de operandos (*Read*). Esto **sí es RAW**: I2 lee (R) algo que I1 todavía no escribió (W) — de ahí el nombre “Read After Write”: la lectura de I2 debería pasar *después* de la escritura de I1 (en el orden lógico), pero el pipeline, sin corrección, haría que ocurra *antes*.

Diagrama de tiempo (pipeline F-D-R-E-W) mostrando el conflicto:

	ciclo:	1	2	3	4	5	
I1: ADD R1,R2,R3			F	D	R	E	W
							<- R1 se escribe recién acá
I2: SUB R4,R1,R5				F	D	R	E
							W
							^
							I2 necesita leer R1 en su etapa R (ciclo 4)
							pero I1 todavía no llegó a su etapa W (ciclo 5)

4.2 WAR — WRITE AFTER READ (ESCRITURA DESPUÉS DE LECTURA)

Ocurre cuando una instrucción posterior **escribe** un registro que una instrucción anterior todavía necesita **leer**, y esa escritura se adelanta indebidamente a la lectura. También se llama **anti-dependencia**.

Ejemplo concreto:

I1: SUB R4, R1, R5	; lee R1
I2: ADD R1, R2, R3	; escribe R1

Si por algún motivo (por ejemplo, ejecución fuera de orden) I2 llegara a escribir R1 *antes* de que I1 termine de leer el R1 original, I1 leería el valor nuevo (incorrecto) en vez del viejo. El nombre WAR describe justamente esto: la *escritura* (de I2) debe respetar el orden y ocurrir *después* de la *lectura* (de I1).

En los pipelines simples de 5 etapas vistos en el curso (donde la etapa *Read* siempre está antes que *Write* para toda instrucción, en orden), WAR y WAW **no pueden ocurrir** naturalmente: como las instrucciones entran y avanzan en orden estricto, I1 (más vieja) siempre llega antes que I2 a cualquier etapa, incluida Write. Estos hazards solo aparecen en diseños más avanzados con **finalización fuera de orden** (ver sección 6) o con operaciones multiciclo (por ejemplo, una división lenta seguida de una suma rápida que la “adelanta”).

4.3 WAW — WRITE AFTER WRITE (ESCRITURA DESPUÉS DE ESCRITURA)

Ocurre cuando dos instrucciones escriben en el mismo registro, y terminan escribiendo **en el orden equivocado** (la más nueva escribe antes que la más vieja), dejando en el registro un valor incorrecto (el de la instrucción vieja, cuando debería quedar el de la nueva).

Ejemplo concreto (instrucciones multiciclo, latencias distintas):

```
I1:  MUL  R1, R2, R3    ; multiplicación, tarda muchos ciclos (ej: 10)
I2:  ADD  R1, R4, R5    ; suma, tarda pocos ciclos (ej: 1)
```

Ambas escriben en R1. Si I2 (más nueva, pero de ejecución rápida) termina y escribe R1 *antes* que I1 (más vieja, pero lenta), el valor final en R1 quedaría siendo el resultado de la multiplicación, sobrescribiendo por error el resultado de la suma — cuando el orden lógico del programa exige que el valor final sea el de I2 (la última instrucción en el programa que escribe R1).

4.4 SOLUCIONES A LOS HAZARDS DE DATOS

1. **Burbuja / stall:** detener el avance de la instrucción que necesita el dato hasta que esté disponible. Simple, pero desperdicia ciclos.
2. **Ejecución fuera de orden (out-of-order execution):** reordenar dinámicamente la ejecución de las instrucciones para que las que no tienen dependencias pendientes avancen mientras las bloqueadas esperan (ver sección 6). Requiere hardware de control mucho más complejo (y mecanismos como *register renaming* para eliminar WAR/WAW).
3. **Register forwarding / bypassing (la solución más elegante para RAW):** en vez de esperar a que el resultado pase por la etapa *Write* y quede guardado en el banco de registros para recién ahí poder leerlo, se agrega un camino de hardware adicional que **toma el resultado apenas sale de la ALU (etapa Execute) y lo entrega directamente a la entrada de la etapa que lo necesita** (típicamente Read/Execute de la siguiente instrucción), sin pasar por el banco de registros.

Cómo funciona el forwarding — diagrama textual con flecha de dependencia:

	ciclo:	1	2	3	4	5
I1: ADD R1,R2,R3		F	D	R	E	W
						(forwarding: el resultado de R1
						sale de la ALU en el ciclo 4
					v	y se "adelanta" directo a I2)
I2: SUB R4,R1,R5			F	D	R	E W
					^	
						I2 usa el valor "adelantado"
						en su propia etapa E (ciclo 5),
						sin esperar a que I1 llegue a W

En lugar de que I2 tenga que leer R1 desde el banco de registros en su etapa *Read* (ciclo 4, cuando el valor todavía no existe), se agrega un multiplexor a la entrada de la ALU que puede tomar el operando **directamente desde la salida de la ALU de la etapa Execute de la instrucción anterior** (ciclo 4 de I1) y entregárselo a la etapa Execute de I2 (ciclo 5), un ciclo antes de que el registro físico R1 se actualice en la etapa Write de I1. De esta forma se evita el stall casi por completo.

Truco de verificación: para saber si el forwarding “alcanza” a resolver un RAW sin necesidad de burbuja, comparar en qué ciclo la instrucción productora *genera* el dato (normalmente al final de su etapa Execute) contra en qué ciclo la instrucción consumidora *lo necesita* (normalmente al principio de su etapa Execute). Si el dato está listo en el mismo ciclo o antes de que se necesite, el forwarding resuelve el hazard sin parar el pipeline; si la instrucción consumidora lo necesitaría *antes* de que exista (por ejemplo, si depende de una carga desde memoria que tarda más), va a hacer falta al menos una burbuja.

Error frecuente: decir que el forwarding “elimina” el hazard RAW en general. En realidad el forwarding **mitiga** el hazard RAW cuando el dato está disponible a tiempo (por ejemplo, viene de una ALU rápida), pero si el dato depende de un acceso lento a memoria (instrucción de carga, “load”), puede seguir haciendo falta al menos una burbuja de un ciclo aunque haya forwarding, porque el dato físicamente no existe todavía en el momento en que se lo necesita.

4.5 TABLA RESUMEN DE HAZARDS DE DATOS

Tipo	Nombre completo	Situación	¿Ocurre en pipeline simple en orden?
RAW	Read After Write	Se lee un dato que aún no fue escrito	Sí, es el más común
WAR	Write After Read	Se escribe un dato antes de que otra instrucción lo lea	No (solo en fuera de orden)
WAW	Write After Write	Dos escrituras terminan en orden incorrecto	No (solo en fuera de orden / multiciclo)

5. Hazards de control

El problema. Los saltos (condicionales o incondicionales) alteran el flujo normal de ejecución. La etapa de *Fetch*, por defecto, siempre asume que la próxima instrucción a leer es la que está justo a continuación en memoria. Esto es **incorrecto si la instrucción actual es un salto**: en el caso de un salto incondicional, siempre está mal; en el caso de un salto condicional, la condición (¿se toma el salto o no?) recién se evalúa varias etapas después del Fetch (normalmente en Execute), por lo que durante esos ciclos intermedios el pipeline ya “adelantó” el Fetch (y quizás el Decode) de instrucciones que **podrían no corresponder** a la secuencia real del programa. Si el salto efectivamente se toma, esas instrucciones adelantadas deben descartarse, perdiendo los ciclos que costó traerlas.

SOLUCIONES

1. **Prefetch de ambos destinos posibles:** se hace el fetch simultáneo de la instrucción siguiente en memoria y de la instrucción en la dirección de destino del salto. Cuando finalmente se sabe si el salto se toma o no, se descarta la que no corresponda y se sigue con la otra sin perder tiempo adicional de fetch.
2. **Múltiples flujos especulativos (multiple streams):** se duplican las etapas del pipeline anteriores a la etapa donde se resuelve el salto, permitiendo avanzar en la ejecución de *ambos* caminos posibles simultáneamente. Cuando se resuelve el salto, se descarta el flujo que no correspondía. Es más costoso en hardware que el prefetch simple, y es propenso a generar además hazards estructurales (duplicar recursos implica más consumo de área/energía).

3. **Salto demorado (delayed branch):** técnica muy usada en procesadores RISC. La idea es que **siempre se ejecuta la instrucción inmediatamente siguiente al salto**, sin importar el resultado del salto (se dice que esa instrucción está en el “delayed slot”). Como esto es una garantía arquitectónica, el compilador (o el programador de bajo nivel) puede **reordenar el código** para colocar en esa posición una instrucción que de todos modos había que ejecutar (por ejemplo, una instrucción independiente que estaba antes del salto y no afecta ni es afectada por la condición del salto). Si no se encuentra una instrucción útil para poner ahí, se coloca un NOP. Esta técnica funciona bien porque el procesador puede resolver el salto en una etapa temprana del pipeline (por ejemplo la segunda etapa) y así no hay penalización: nunca hay que “tirar” trabajo, porque el delayed slot siempre se ejecuta de todos modos.
4. **Predicción de salto (branch prediction):** aplica a saltos condicionales. Consiste en “adivinar” de antemano si el salto se va a tomar o no, y seguir haciendo fetch en base a esa predicción. Si se acierta, no hay penalización (el pipeline nunca tuvo que descartar nada). Si se falla, hay que vaciar las etapas que avanzaron con la predicción incorrecta. Puede ser:
 - **Estática:** la predicción se decide de antemano y no cambia con la ejecución (ejemplos: asumir que el salto nunca se toma; asumir que siempre se toma — ya que estadísticamente más del 50% de los saltos condicionales sí se toman; o basarse en el tipo de condición codificada en la instrucción).
 - **Dinámica:** la predicción se ajusta en tiempo de ejecución según el comportamiento reciente del mismo salto, típicamente usando una **Tabla de Historia de Saltos (BHT)** que recuerda si ese salto en particular se tomó o no las últimas veces, y una máquina de estados que solo cambia la predicción si falla dos veces seguidas (para no ser demasiado sensible a un comportamiento puntual atípico).

Truco de verificación: para saber si una técnica de manejo de saltos “cuesta” ciclos, pregúntate: ¿el hardware alguna vez tiene que **descartar trabajo ya hecho**? El salto demorado, bien usado (delayed slot lleno con instrucción útil), **nunca** descarta nada — por eso no tiene penalización. El prefetch de ambos destinos y la predicción de salto **sí** pueden llegar a descartar trabajo (cuando la predicción falla o cuando finalmente se sabe cuál de los dos caminos no correspondía).

Error frecuente: confundir “salto demorado” con “predicción de salto”. Son técnicas distintas y hasta complementarias: el salto demorado es una **garantía arquitectónica** (siempre se ejecuta la instrucción del delayed slot, no hay apuestas), mientras que la predicción de salto es una **apuesta** que puede fallar y generar descarte de instrucciones ya buscadas/decodificadas.

6. Ejecución fuera de orden

Idea general. En vez de forzar a que las instrucciones avancen por el pipeline estrictamente en el orden en que aparecen en el programa, un procesador con ejecución fuera de orden mantiene un conjunto de instrucciones ya leídas de memoria (una especie de “sala de espera” o *reservation station*), y las va marcando como “**prontas para ejecutar**” o no, según si sus dependencias de datos y de control ya fueron satisfechas. Apenas una instrucción está pronta, puede pasar a ejecutarse — **sin importar si hay instrucciones anteriores del programa que todavía están esperando**. A medida que se generan nuevos resultados, se actualiza el estado de las instrucciones que esperaban por ellos, liberando a más instrucciones para ejecutar.

Por qué ayuda con los hazards de datos sin parar el pipeline por completo: con un pipeline simple en orden, si la instrucción I2 depende de un resultado que I1 todavía no calculó, *todo* el pipeline detrás de I2 se detiene (burbuja), aunque hubiera instrucciones I3, I4 más adelante en el programa que **no** tuvieran ninguna dependencia con I1 y perfectamente podrían ejecutarse mientras tanto. La ejecución fuera de orden aprovecha exactamente esa situación: en vez de bloquear todo el pipeline esperando a I1, deja avanzar a I3 o I4 (que sí están listas) mientras I2 sigue esperando. Esto mantiene ocupadas las unidades funcionales (la ALU, por ejemplo) en vez de dejarlas ociosas, mejorando el throughput real.

Importante: aunque la *ejecución* puede reordenarse, el resultado final del programa tiene que ser exactamente el mismo que si se hubiera ejecutado en orden estricto. Para lograrlo, los procesadores modernos agregan una etapa de **commit** (o *retirement*) que se asegura de que los efectos visibles (escritura final en registros/memoria) sí se apliquen en el orden lógico del programa, aunque el trabajo interno se haya hecho fuera de orden. Esta es la base de los diseños **superscalares** modernos (tema que excede el temario de este módulo, y que el apunte del curso solo menciona brevemente en un capítulo aparte).

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	Pipeline de $k=4$ etapas, se ejecutan $n=8$ instrucciones. Calcular SS .	$T_{\text{sin}}=32$, $T_{\text{con}}=11$, $SS=32/11=2.909$
2	Pipeline de $k=6$ etapas, $n \rightarrow \infty$. ¿Cuál es el speedup límite?	$SS \rightarrow k = 6$
3	¿Cuántos ciclos tarda en completarse la <i>primera</i> instrucción en un pipeline de k etapas?	k ciclos (no hay atajo, tiene que atravesar todas las etapas)
4	ADD R1, R2, R3 seguido de MUL R5, R1, R4. ¿Qué tipo de hazard hay?	RAW (la segunda lee R1, que la primera aún no escribió)
5	Dos instrucciones I1 (vieja, lenta) e I2 (nueva, rápida) escriben el mismo registro y I2 termina antes que I1. ¿Qué hazard es?	WAW (Write After Write)
6	¿En qué etapa se genera normalmente el dato que se “adelanta” con forwarding?	Al final de la etapa Execute (salida de la ALU)
7	¿El salto demorado puede generar descarte de instrucciones ya buscadas?	No, por diseño siempre se ejecuta la instrucción del delayed slot
8	Un hazard estructural, ¿es un problema de datos o de hardware compartido?	De hardware compartido (dos etapas compiten por el mismo recurso físico)
9	¿Cuál es la solución “universal” (aunque poco eficiente) para cualquier hazard?	Insertar una burbuja / stall (esperar)
10	Pipeline $k=5$, $n=20$. Calcular SS .	$T_{\text{sin}}=100$, $T_{\text{con}}=24$, $SS=100/24=4.167$

Resumen ultra-rápido (2 minutos)

- **Pipeline:** divide la ejecución en k etapas independientes de hardware y las solapa entre instrucciones consecutivas. No acelera una instrucción individual; acelera el throughput.
- **Speedup:** $SS = \frac{nk}{n+k-1}$. Sin pipeline: nk ciclos. Con pipeline: $n+k-1$ ciclos (la primera instrucción tarda k , después sale una por ciclo). Cuando $n \rightarrow \infty$, $SS \rightarrow k$ (el máximo teórico).
- **Hazard estructural:** dos etapas pelean por el mismo hardware (ej: memoria compartida entre Fetch y Read). Se resuelve con burbuja o duplicando hardware.

- **Hazard de datos:** RAW (leer antes de que se escriba — el más común), WAR (escribir antes de que se lea — anti-dependencia), WAW (dos escrituras en orden incorrecto). Soluciones: burbuja, ejecución fuera de orden, o **register forwarding** (adelantar el resultado desde la ALU directo a quien lo necesita, sin pasar por el banco de registros).
- **Hazard de control:** los saltos rompen el supuesto de “la próxima instrucción es la siguiente en memoria”. Soluciones: prefetch de ambos destinos, múltiples flujos especulativos, salto demorado (siempre se ejecuta la instrucción del delayed slot, reordenada por el compilador para que sea útil), predicción de salto (estática o dinámica, con tablas de historia).
- **Ejecución fuera de orden:** deja avanzar instrucciones “prontas” aunque haya instrucciones anteriores bloqueadas por dependencias, manteniendo ocupadas las unidades funcionales; requiere un mecanismo de *commit* en orden para que el resultado final sea correcto.